



Understanding **React**

THE BASELINE

Vanilla JavaScript

- ✗ Imperative: Micromanaging the DOM step-by-step.
- ✗ State Syncing: UI easily falls out of sync with data.
- ✗ Spaghetti Code: Event listeners tangled with markup.

// Imperative UI Updates

```
const btn = document.getElementById('btn');
const label = document.getElementById('count');
let count = 0;
btn.addEventListener('click', () => {
  count++;
  label.innerText = `Count: ${count}`;
  if (count > 5) {
    label.classList.add('warning');
  }
});
```

The React Philosophy

Solving the DOM manipulation problem.



Declarative

Define the end state, let React handle DOM updates.



Component-Based

Break UI into reusable, self-contained functions.



Reactive

UI automatically re-renders when data changes.

THE BUILDING BLOCKS

Components



Standard JavaScript functions.



Return HTML-like syntax.



Capitalized function names.

```
// It's just a function returning UI
export default function ProfileCard() {
  return (
    <div className="card">
      
      <h2>Alice Engineer</h2>
      <p>Frontend Developer</p>
    </div>
  );
}
```

HTML vs. JSX

JavaScript XML: HTML syntax supercharged by JS.

Standard HTML

```
<!-- class attribute -->
<div class="container">

  <!-- unclosed tags allowed -->
  
  <input type="text">

  <!-- inline styles as string -->
  <p style="color: red; font-size: 14px;">
    Static text
  </p>

</div>
```

React JSX

```
{/* className (JS keyword collision) */}
<div className="container">

  {/* MUST self-close tags */}
  
  <input type="text" />

  {/* inline styles as JS objects */}
  <p style={{ color: 'red', fontSize: 14 }}>
    {dynamicJavascriptVariable}
  </p>

</div>
```

// 1. Parent passes data like HTML attributes

```
function App() {  
  return (  
    <main>  
      <UserBadge role="Admin" active={true} />  
      <UserBadge role="Guest" active={false} />  
    </main>  
  );  
}
```

// 2. Child receives data as function arguments

```
function UserBadge({ role, active }) {  
  return <span className="badge">{role}</span>;  
}
```

DATA FLOW

Props

Function Arguments

Standard JS parameters passed as objects.

Read-Only

Children cannot modify props.

Top-Down Flow

Parent to child transmission only.

Event Handling

Passing functions to JSX elements.

Vanilla DOM

```
<!-- lowercase, string evaluation -->
<button onclick="handleClick()">
  Click Me
</button>

<script>
  function handleClick() {
    alert('Vanilla JS');
  }
</script>
```

React Synthetic Events

```
export default function App() {
  // Define handler
  const handleClick = () => {
    alert('React Event');
  };

  /* camelCase, pass function reference */
  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

INTERACTIVITY

Component State

Memory that triggers re-renders.

- Mutable local data.
- Calling setter re-runs the function.
- Preserved between render cycles.

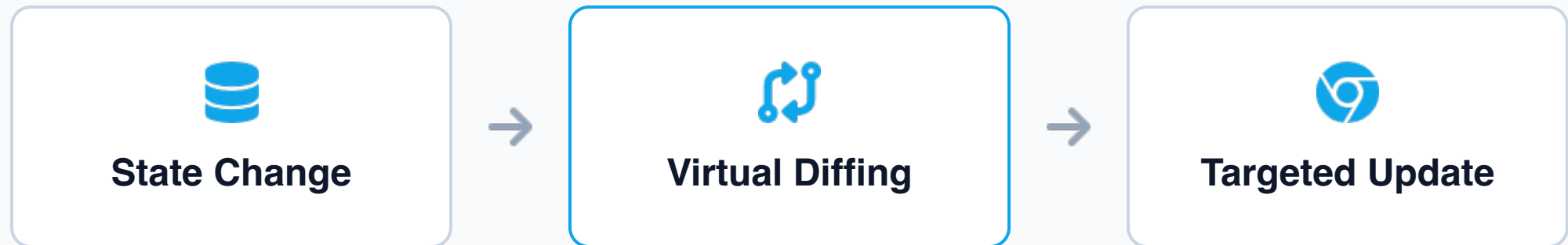
```
import { useState } from 'react';

export default function Counter() {
  // [currentValue, setterFunction]
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicks: {count}
    </button>
  );
}
```

The Virtual DOM

Optimizing browser rendering through memory diffing.



Conditional Rendering

```
// Utilizing standard JS logical operators
function Dashboard({ isLoggedIn, hasUnreadMsgs }) {
  return (
    <div>
      {/* Ternary operator for Either/Or */}
      {isLoggedIn ? <UserPanel /> : <LoginPrompt />}

      {/* Logical AND (&&) for Show/Hide */}
      {hasUnreadMsgs && (
        <span className="alert">New Messages!</span>
      )}
    </div>
  );
};
```

```
const users = [  
  { id: 1, name: 'Alice' },  
  { id: 2, name: 'Bob' }  
];  
  
function UserList() {  
  return (  
    <ul>  
      {users.map(user => (  
        /* Keys are mandatory for loops */  
        <li key={user.id}>{user.name}</li>  
      ))}  
    </ul>  
  );  
}
```

Lists & Keys

Array.map()

Transform data arrays into JSX arrays.

The 'key' Prop

Unique identifier for Virtual DOM diffing.

Performance

Prevents unnecessary re-rendering of lists.

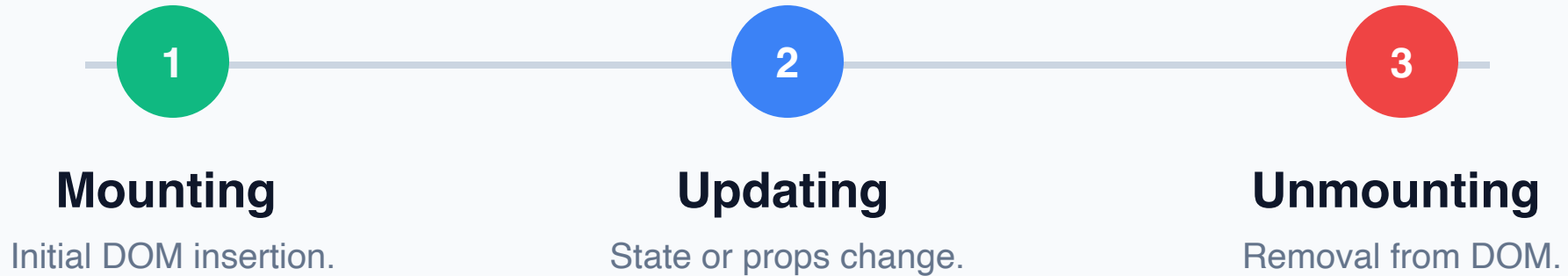
Controlled Components

React state as the single source of truth for forms.

```
function SearchForm() {  
  const [query, setQuery] = useState('');  
  
  return (  
    <form onSubmit={e => e.preventDefault()}>  
      <input  
        value={query} {/* State drives input */}  
        onChange={e => setQuery(e.target.value)}  
      />  
      <p>Searching for: {query}</p>  
    </form>  
  );  
}
```

The Component Lifecycle

Phases of a component's existence.



HOOKS

useEffect

Synchronizing components with external systems.

- ↔ Data fetching requests.
- ↔ Direct DOM manipulations.
- ↔ Third-party subscriptions.

```
// Runs after every render
useEffect(() => {
  document.title = `Updated title`;
});
```

```
// Runs ONLY on mount (empty array)
useEffect(() => {
  fetchData();
}, []);
```

```
// Runs when 'userId' changes
useEffect(() => {
  loadProfile(userId);
}, [userId]);
```

Context API

Bypassing prop drilling.

- Global state management.
- Teleport data deep into tree.
- Avoids passing props unnecessarily.

```
const ThemeContext = createContext('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar /> { /* Child uses useContext() */ }
    </ThemeContext.Provider>
  );
}
```

```
// Extracting JS logic for reuse
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return width;
}
```

Custom Hooks

Logic Reuse

Share stateful logic between components.

Standard JS Functions

Must start with 'use'.

Thinking in React

Mental model for app architecture.

1. Breakdown Identify UI hierarchy visually.

2. Static Build hardcoded visual version.

3. State Find minimal state representation.

4. Flow Implement inverse data flow.

Beyond Local State

Ecosystem tools for complex applications.



Redux

Strict unidirectional data flow.

Zustand

Minimal, hook-based store management.



React Query

Server state synchronization.

The Meta-Frameworks

Production-ready React environments.

Next.js

Full-stack SSR and static generation.

Remix

Web standards focused routing.

React Native

Native mobile app compilation.

Vite

Lightning fast local development.

React Essentials Recap

UI

Functions returning JSX

Props

Top-down arguments

State

Mutable local memory

Effects

External sync



Time to Code

```
npm create vite@latest my-app -- --template react
```

npm create vite@latest: Fetches the latest version of the Vite scaffolding tool.

my-app: Creates a new folder with this name for your project.

--: Tells npm to pass the following arguments directly to Vite.

--template react: Bypasses the interactive menu and instantly generates a React

environment using standard JavaScript. *(Note: you can use `--template react-ts` if you want TypeScript).*